COL334 · COMPUTER NETWORKS · ASSIGNMENT 3

Routing, TCP over raw sockets and HTTP

Report

Team chill_chill · Shivanshu Aryan (2024CS10237) · Nancy Nagar (2024CS10518)

Setup

We built the network from four Ubuntu VMs, *client*, *r1*, *r2* and *server*, running in UTM (QEMU). UTM has no named internal networks, so each of the four links is a point-to-point QEMU socket netdev on the loopback interface (`lanA`, `lanB`, `lanC`, `linkD`), with no DHCP server on any of them. Each VM also has one management adapter, `enp0s1`, that we use for SSH access; none of our scripts touches it and none of the experiments use it. Before running anything, we set `rp_filter` to 1 on the interfaces of the four lab networks on every VM, then ran that machine's script from `netcfg/` once as root.

VM	Links (socket netdevs)	Script
client	LAN A, Link D	`netcfg/client.sh IF_A IF_D`
r1	LAN A, LAN B	`netcfg/r1.sh IF_A IF_B`
r2	LAN B, LAN C, Link D	`netcfg/r2.sh IF_B IF_C IF_D`
server	LAN C	`netcfg/server.sh IF_C`

Our TCP runs on a raw socket, so the kernel has no socket on our ports and would reply to every SYN or SYN-ACK with its own RST. To stop that, we add one iptables rule by hand on the client and the server. It is not part of `netcfg` or of our code, and the server is always started on a port in 8080–8083.

```
client$ sudo iptables -A OUTPUT -p tcp --tcp-flags RST RST --sport 61000:65535 -j DROP
server$ sudo iptables -A OUTPUT -p tcp --tcp-flags RST RST --sport 8080:8083 -j DROP
```

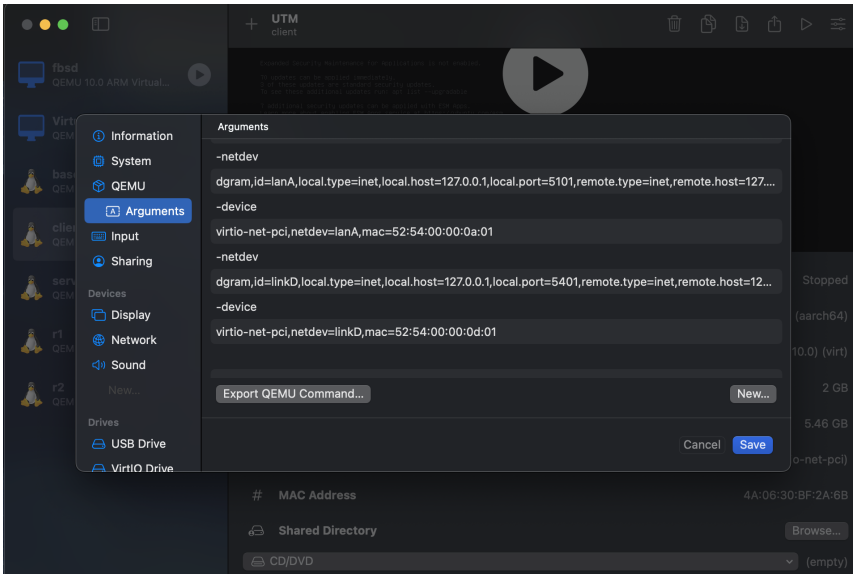
R1 Adapters, addresses and routes

What we did. For each VM we opened UTM's QEMU Arguments pane, which lists every `-netdev`/`-device` pair and so shows which links the VM has, then ran `ip -br addr` and `ip route` inside the VM after its script had run. The two together show that each machine sits on exactly the networks it should, with the addresses from the plan.

Machine	Addresses	Routes added by the script	Forwarding
client	10.10.1.10/24, 10.10.4.2/30	`default via 10.10.1.1`	off
r1	10.10.1.1/24, 10.10.2.1/24	`default via 10.10.2.2`	on
r2	10.10.2.2/24, 10.10.3.1/24, 10.10.4.1/30	`10.10.1.10/32 via 10.10.4.2` `10.10.1.0/24 via 10.10.2.1`	on
server	10.10.3.10/24	`default via 10.10.3.1`	off

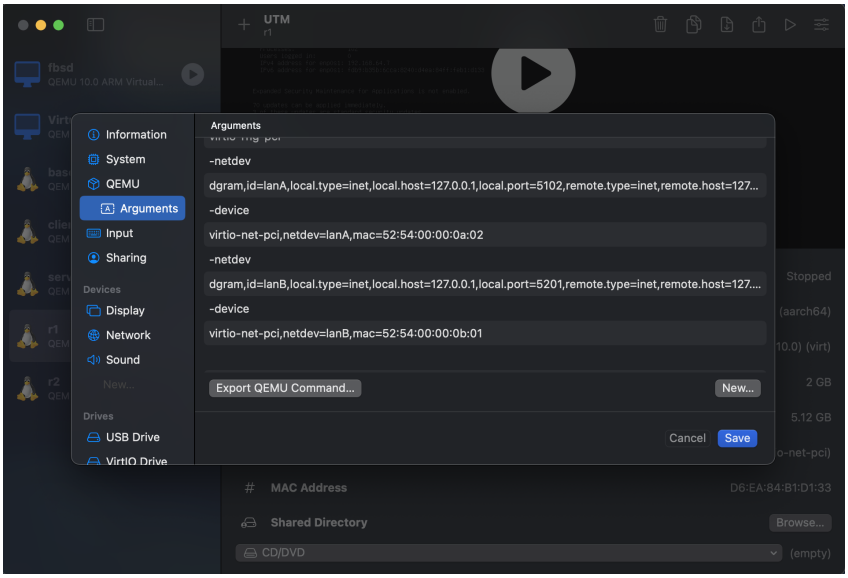
Table 1. What each script configures. Routes listed are in addition to the connected networks.

Why these routes. The client only has a default route via r1, so everything it sends goes client → r1 → r2 → server, with source address 10.10.1.10. r1 needs a single route, a default via r2, because both LAN C and Link D sit behind r2. On r2 the host route for 10.10.1.10 beats the /24 by longest-prefix match, so everything addressed to the client, including all the server's replies, leaves over Link D. The rest of LAN A (r1 itself) is still reached through r1. r2 has no default route, and the server only needs a default via r2.



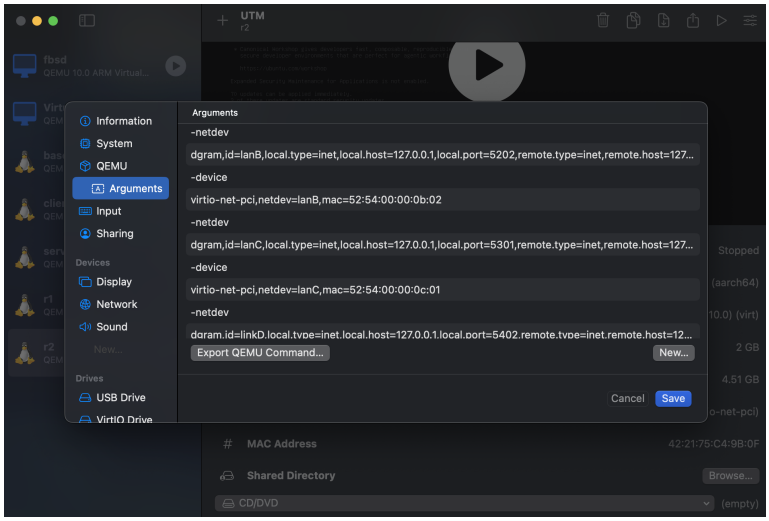

```
nancy@client:~/Network_Stack$ ip -br addr; ip route
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s1           UP             192.168.64.5/24 metric 100 fdb9:b35b:6cca:8240:4
806:30ff:febf:2a6b/64 fe80::4806:30ff:febf:2a6b/64
enp0s9           UP             10.10.1.10/24 fe80::5054:ff:fe00:a01/64
enp0s10          UP             10.10.4.2/30 fe80::5054:ff:fe00:d01/64
default via 10.10.1.1 dev enp0s9
default via 192.168.64.1 dev enp0s1 proto dhcp src 192.168.64.5 metric 100
10.10.1.0/24 dev enp0s9 proto kernel scope link src 10.10.1.10
10.10.4.0/30 dev enp0s10 proto kernel scope link src 10.10.4.2
192.168.64.0/24 dev enp0s1 proto kernel scope link src 192.168.64.5 metric 100
192.168.64.1 dev enp0s1 proto dhcp scope link src 192.168.64.5 metric 100
nancy@client:~/Network_Stack$
```

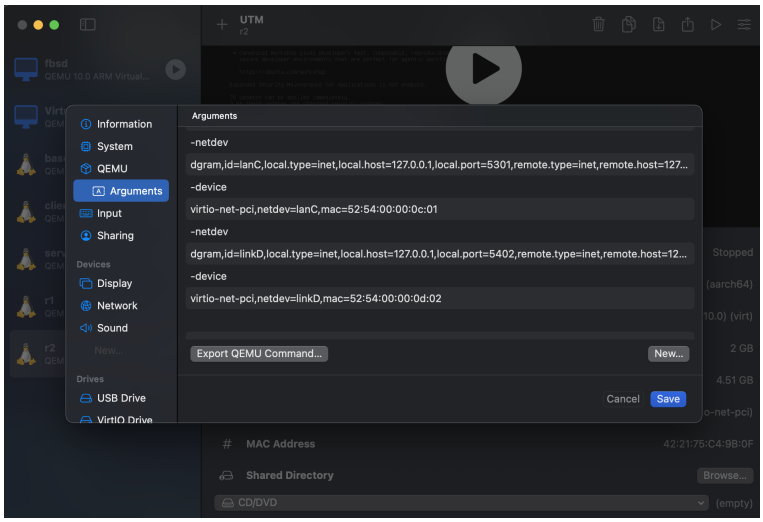
Figure 1. client: QEMU arguments (top), then `ip -br addr` and `ip route` (bottom).




```
nancy@r1:~/Network_Stack$ ip -br addr; ip route
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s1           UP             192.168.64.7/24 metric 100 fdb9:b35b:6cca:8240:d
4ea:84ff:feb1:d133/64 fe80::d4ea:84ff:feb1:d133/64
enp0s9           UP             10.10.1.1/24 fe80::5054:ff:fe00:a02/64
enp0s10          UP             10.10.2.1/24 fe80::5054:ff:fe00:b01/64
default via 10.10.2.2 dev enp0s10
default via 192.168.64.1 dev enp0s1 proto dhcp src 192.168.64.7 metric 100
10.10.1.0/24 dev enp0s9 proto kernel scope link src 10.10.1.1
10.10.2.0/24 dev enp0s10 proto kernel scope link src 10.10.2.1
192.168.64.0/24 dev enp0s1 proto kernel scope link src 192.168.64.7 metric 100
192.168.64.1 dev enp0s1 proto dhcp scope link src 192.168.64.7 metric 100
nancy@r1:~/Network_Stack$
```

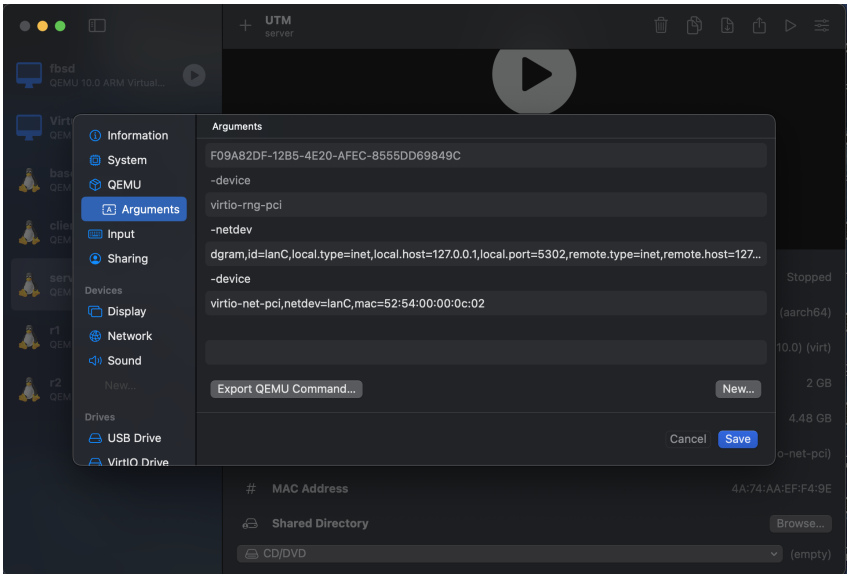
Figure 2. r1: QEMU arguments (top), then `ip -br addr` and `ip route` (bottom).






```
nancy@r2:~/Network_Stack$ ip -br addr; ip route
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s1           UP             192.168.64.8/24 metric 100 fdb9:b35b:6cca:8240:4
021:75ff:fec4:9b0f/64 fe80::4021:75ff:fec4:9b0f/64
enp0s9           UP             10.10.2.2/24 fe80::5054:ff:fe00:b02/64
enp0s10          UP             10.10.3.1/24 fe80::5054:ff:fe00:c01/64
enp0s11          UP             10.10.4.1/30 fe80::5054:ff:fe00:d02/64
default via 192.168.64.1 dev enp0s1 proto dhcp src 192.168.64.8 metric 100
10.10.1.0/24 via 10.10.2.1 dev enp0s9
10.10.1.10 via 10.10.4.2 dev enp0s11
10.10.2.0/24 dev enp0s9 proto kernel scope link src 10.10.2.2
10.10.3.0/24 dev enp0s10 proto kernel scope link src 10.10.3.1
10.10.4.0/30 dev enp0s11 proto kernel scope link src 10.10.4.1
192.168.64.0/24 dev enp0s1 proto kernel scope link src 192.168.64.8 metric 100
192.168.64.1 dev enp0s1 proto dhcp scope link src 192.168.64.8 metric 100
nancy@r2:~/Network_Stack$
```

Figure 3. r2: QEMU arguments, scrolled to show all three links (top and middle), then `ip -br addr` and `ip route` (bottom).




```
nancy@server:~/Network_Stack$ ip -br addr; ip route
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s1           UP             192.168.64.6/24 metric 100 fdb9:b35b:6cca:8240:4874
:aaff:feef:f49e/64 fe80::4874:aaff:feef:f49e/64
enp0s9           UP             10.10.3.10/24 fe80::5054:ff:fe00:c02/64
default via 10.10.3.1 dev enp0s9
default via 192.168.64.1 dev enp0s1 proto dhcp src 192.168.64.6 metric 100
10.10.3.0/24 dev enp0s9 proto kernel scope link src 10.10.3.10
192.168.64.0/24 dev enp0s1 proto kernel scope link src 192.168.64.6 metric 100
192.168.64.1 dev enp0s1 proto dhcp scope link src 192.168.64.6 metric 100
nancy@server:~/Network_Stack$
```

Figure 4. server: QEMU arguments (top), then `ip -br addr` and `ip route` (bottom).

What the screenshots show. The client has `enp0s9` = 10.10.1.10/24 (LAN A) and `enp0s10` = 10.10.4.2/30 (Link D), with `default via 10.10.1.1 dev enp0s9`. r1 has `enp0s9` = 10.10.1.1/24 and `enp0s10` = 10.10.2.1/24, and one route besides the connected networks, `default via 10.10.2.2 dev enp0s10`. r2 has `enp0s9` = 10.10.2.2/24, `enp0s10` = 10.10.3.1/24 and `enp0s11` = 10.10.4.1/30, with `10.10.1.10 via 10.10.4.2 dev enp0s11` and `10.10.1.0/24 via 10.10.2.1 dev enp0s9`. The server has `enp0s9` = 10.10.3.10/24 and `default via 10.10.3.1 dev enp0s9`. These match Table 1 of the handout address for address. The QEMU arguments show the links as `lanA`, `lanB`, `lanC` and `linkD`, and each VM has exactly the netdev/device pairs listed in the setup table.
Every VM also lists `enp0s1`, the management adapter, with a `192.168.64.x` address and the default route that DHCP added for it. That route has metric 100, while the routes our scripts install keep the default metric, so all traffic between the 10.10.x networks follows the routes above. The scripts themselves install no default route on r2.

R2 traceroute in both directions

What we did. We ran `traceroute -n` from the client to the server and from the server to the client. The two traces are different, and that is what the routing was designed to produce.

```
client$ traceroute -n 10.10.3.10        server$ traceroute -n 10.10.1.10
 1  10.10.1.1    (r1)                    1  10.10.3.1    (r2)
 2  10.10.4.1    (r2, Link D address)    2  10.10.1.10   (client)
 3  10.10.3.10   (server)
```

Client to server. *Hop 1:* the probe with TTL 1 dies at r1, the client's gateway, which answers from its LAN A address, 10.10.1.1. *Hop 2:* r1 forwards the TTL 2 probe to r2, where it expires. r2 sends the ICMP error to 10.10.1.10, and Linux picks the source address of that error from the route it uses to send it, not from the interface the probe came in on. r2's route to 10.10.1.10 is the host route over Link D, so the error leaves on

that interface and carries r2's Link D address, 10.10.4.1. That is why r2 shows up as 10.10.4.1 rather than as 10.10.2.2, the address the probe actually reached. The client accepts it because 10.10.4.1 is directly connected on Link D. *Hop 3:* the probe reaches the server, which answers with a port-unreachable from 10.10.3.10. That reply travels back through r2 and over Link D as well.

Server to client. *Hop 1:* r2 is the server's gateway, so the TTL 1 probe dies there and the error comes back from 10.10.3.1. *Hop 2:* r2 forwards the next probe straight over Link D to the client, and the client answers from the probed address, 10.10.1.10. r1 never appears, since it is not on this path.

Why the directions differ. The client's only non-connected route goes via r1, while r2 sends everything for the client over Link D. So the forward path has three hops and the reverse path has two, and r2 appears with its Link D address in the forward trace because its replies take the reverse path.

```
nancy@client:~/Network_Stack$ traceroute -n 10.10.3.10
traceroute to 10.10.3.10 (10.10.3.10), 30 hops max, 60 byte packets
 1  10.10.1.1  2.462 ms  2.200 ms  2.244 ms
 2  10.10.4.1  4.919 ms  5.151 ms  5.144 ms
 3  10.10.3.10  6.999 ms  7.053 ms  7.044 ms
```

Figure 5. `traceroute -n 10.10.3.10` on the client.

```
nancy@server:~/Network_Stack$ traceroute -n 10.10.1.10
traceroute to 10.10.1.10 (10.10.1.10), 30 hops max, 60 byte packets
 1  10.10.3.1  1.471 ms  1.373 ms  1.367 ms
 2  10.10.1.10  3.954 ms  3.948 ms  3.944 ms
```

Figure 6. `traceroute -n 10.10.1.10` on the server.

> **What the screenshots show.** Client to server has three hops: 10.10.1.1 at about 2.2–2.5 ms, 10.10.4.1 at about 5.0 ms and 10.10.3.10 at about 7.0 ms. Server to client has two: 10.10.3.1 at about 1.4 ms and 10.10.1.10 at about 3.9 ms. The second hop of the forward trace is the Link D address of r2, exactly as explained above, and r1 is absent from the reverse trace. The round-trip times grow with each hop, and the reverse round trip to the client (about 3.9 ms) is clearly shorter than the forward one to the server (about 7.0 ms), which fits the shorter reverse path.

R3 Reverse-path filtering

What we did. We read back every `rp_filter` value with `sysctl -a | grep rp_filter` on all four machines. On the VMs the interface names are `enp0s9` and `enp0s10` on the client (IF_A and IF_D), `enp0s9` and `enp0s10` on r1 (IF_A and IF_B), `enp0s9`, `enp0s10` and `enp0s11` on r2 (IF_B, IF_C and IF_D) and `enp0s9` on the server (IF_C). The management adapter `enp0s1` and `lo` show 2 in the output, which is Ubuntu's default; the scripts do not touch them. The kernel uses max(`conf.all`, `conf.<if>`) for each interface. All machines start at 1 and our scripts never touch `conf.all`, so it stays at 1. That means setting an interface to 0 would change nothing, and the only way to relax one is loose mode (2). Loose mode accepts a packet if its source is reachable through *any* interface, while strict mode needs the best route back to the source to leave through the interface the packet arrived on.

Machine	Interface	Value	Why
client	IF_A	1	Sources seen here are on LAN A or behind r1, and they route back via IF_A.
client	IF_D	**2**	Replies from the server arrive here with sources like 10.10.3.10, but the client's route to that network is the default via IF_A. Strict mode would drop every reply.
r1	both	1	Must stay strict, and it works: IF_A only sees LAN A sources, and IF_B sees LAN B and LAN C sources, which both route back via IF_B. It is also why the client sends from 10.10.1.10 and not 10.10.4.2.
r2	IF_B	**2**	Client packets arrive here from r1 with source 10.10.1.10, but r2's best route to that address is over IF_D. Strict mode would drop the entire forward direction.
r2	IF_C, IF_D	1	IF_C only sees LAN C sources, and IF_D only sees the directly connected 10.10.4.2.
server	IF_C	1	Must stay strict. It has a single interface, so every source routes back through it.

Table 2. The `rp_filter` value on each interface. Loose mode is used only where the asymmetric routing needs it.

```
nancy@client:~/Network_Stack$ sysctl -a 2>/dev/null | grep '\.rp_filter'
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
net.ipv4.conf.enp0s1.rp_filter = 2
net.ipv4.conf.enp0s10.rp_filter = 2
net.ipv4.conf.enp0s9.rp_filter = 1
net.ipv4.conf.lo.rp_filter = 2
```

```
nancy@r1:~/Network_Stack$ sysctl -a 2>/dev/null | grep '\.rp_filter'
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
net.ipv4.conf.enp0s1.rp_filter = 2
net.ipv4.conf.enp0s10.rp_filter = 1
net.ipv4.conf.enp0s9.rp_filter = 1
net.ipv4.conf.lo.rp_filter = 2
```

```
nancy@r2:~/Network_Stack$ sysctl -a 2>/dev/null | grep '\.rp_filter'
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
net.ipv4.conf.enp0s1.rp_filter = 2
net.ipv4.conf.enp0s10.rp_filter = 1
net.ipv4.conf.enp0s11.rp_filter = 1
net.ipv4.conf.enp0s9.rp_filter = 2
net.ipv4.conf.lo.rp_filter = 2
```

```
nancy@server:~/Network_Stack$ sysctl -a 2>/dev/null | grep '\.rp_filter'
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
net.ipv4.conf.enp0s1.rp_filter = 2
net.ipv4.conf.enp0s9.rp_filter = 1
net.ipv4.conf.lo.rp_filter = 2
```

Figure 7. `rp_filter` values on client, r1, r2 and server (top to bottom).

What the screenshots show. On every machine `conf.all` and `conf.default` are 1. On the client `enp0s9` (IF_A) is 1 and `enp0s10` (IF_D) is 2. r1 has 1 on both `enp0s9` and `enp0s10`. r2 has 2 on `enp0s9` (IF_B) and 1 on `enp0s10` and `enp0s11`. The server has 1 on `enp0s9`. So strict mode is kept on r1 and the server as required, and loose mode appears only on the client's Link D interface and on r2's LAN B interface, which are the two places where the asymmetric path would otherwise be dropped.

R4 Our client against a standard web server

What we did. On the server VM we served a directory with `python3 -m http.server 8000`. It holds an HTML page and a random 300 000-byte binary file. From the client we fetched each file with our own client and printed its exit code, then fetched the same URL with `curl` and compared the two `md5sum`s. We expect exit code 0 for both downloads and identical checksums for each pair.

```
server$ cd ~/r4 && python3 -m http.server 8000
client$ sudo ./client/run-client http://10.10.3.10:8000/index.html -o a.html; echo $?
client$ curl -s -o a_curl.html http://10.10.3.10:8000/index.html; md5sum a.html a_curl.html
```

```
nancy@client:~/Network_Stack$ cd ~/Network_Stack
sudo ./client/run-client http://10.10.3.10:8000/index.html -o a.html; echo $?
curl -s -o a_curl.html http://10.10.3.10:8000/index.html
md5sum a.html a_curl.html
[sudo] password for nancy:
HTTP/1.0 200 OK
0
c992ecc7298f2af44a300cefd26bb6ca  a.html
c992ecc7298f2af44a300cefd26bb6ca  a_curl.html
```

Figure 8. HTML page downloaded with our client: exit code, and `md5sum` compared with curl.

```
nancy@client:~/Network_Stack$ sudo ./client/run-client http://10.10.3.10:8000/bi
n300k.bin -o b.bin; echo $?
curl -s -o b_curl.bin http://10.10.3.10:8000/bin300k.bin
md5sum b.bin b_curl.bin
ls -l b.bin
HTTP/1.0 200 OK
0
605a41badc9240fb248d2184096a2ebf  b.bin
605a41badc9240fb248d2184096a2ebf  b_curl.bin
-rw-r--r-- 1 root root 300000 Oct 12 15:09 b.bin
```

Figure 9. 300 KB binary downloaded with our client: exit code, and `md5sum` compared with curl.

> **What the screenshots show.** Both downloads print the status line our client received, `HTTP/1.0 200 OK` from `http.server`, and exit with code 0. For the HTML page the `md5sum` of our file and of curl's file is `c992ecc7298f2af44a300cefd26bb6ca`. For the binary it is `605a41badc9240fb248d2184096a2ebf` for both, and `ls -l` shows `b.bin` is 300 000 bytes. The same binary, with the same checksum, is the file used in the next experiment.

R5 curl, wget and our client against our server

What we did. On the server VM we started our own server with the binary file in its document root and printed the file's `md5sum` there. From the client we downloaded the same file with `curl`, `wget` and our client, one after the other because our server handles one connection at a time, and checked that all three checksums equal the original.

```
server$ md5sum ~/www/bin300k.bin; sudo ./server/run-server -p 8080 -d ~/www
client$ curl -s -o c.bin http://10.10.3.10:8080/bin300k.bin; echo $?
client$ wget -q -O w.bin http://10.10.3.10:8080/bin300k.bin; echo $?
client$ sudo ./client/run-client http://10.10.3.10:8080/bin300k.bin -o o.bin; echo $?
client$ md5sum c.bin w.bin o.bin
```

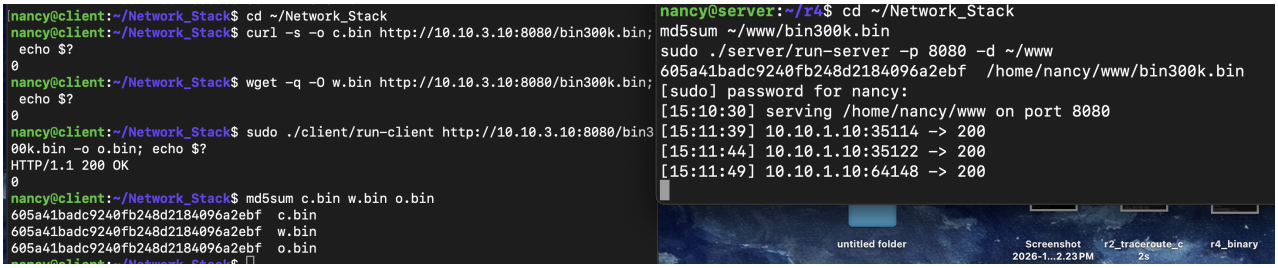


Figure 10. curl, wget and our client downloading the same file from our server, with the original `md5sum`.

What the screenshots show. The original file on the server has `md5sum` `605a41badc9240fb248d2184096a2ebf`, and `c.bin` (curl), `w.bin` (wget) and `o.bin` (our client) all have exactly that checksum. All three commands exit with 0, and our client prints `HTTP/1.1 200 OK`, the status line from our own server. The server's log shows three requests, all from 10.10.1.10 and all answered with 200. The first two come from ordinary kernel ports (35114 and 35122, curl and wget). The third comes from port 64148, which is inside the 61000–65535 range our client draws from.

R6 Packet capture and TCP design

What we did. We captured one complete connection of our client with `tcpdump -nn` on the server, where both directions pass through a single interface (on the client, outgoing packets use one interface and replies come back on another). The client fetched a small file from our server, so the three-way handshake, the request, the response data and ACKs, and the FIN exchange all fit on one screen.

```
server$ sudo ./server/run-server -p 8080 -d ~/www
server$ sudo tcpdump -nn -i IF_C 'tcp port 8080'
client$ sudo ./client/run-client http://10.10.3.10:8080/index.html -o x.html
```

```
^Cnancy@server:~/Network_Stacksudo tcpdump -nn -i enp0s9 'tcp port 8080'0'
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on enp0s9, link-type EN10MB (Ethernet), snapshot length 262144 bytes
15:25:31.477402 IP 10.10.1.10.64980 > 10.10.3.10.8080: Flags [S], seq 1534331092, win 65535, options [mss 1460], length 0
15:25:31.479965 IP 10.10.3.10.8080 > 10.10.1.10.64980: Flags [S.], seq 1058318432, ack 1534331093, win 65535, options [mss 1460], length 0
15:25:31.481034 IP 10.10.1.10.64980 > 10.10.3.10.8080: Flags [.], ack 1, win 65535, length 0
15:25:31.481072 IP 10.10.1.10.64980 > 10.10.3.10.8080: Flags [P.], seq 1:118, ack 1, win 65535, length 117: HTTP: GET /index.html HTTP/1.1
15:25:31.481537 IP 10.10.3.10.8080 > 10.10.1.10.64980: Flags [.], ack 118, win 65535, length 0
15:25:31.482753 IP 10.10.3.10.8080 > 10.10.1.10.64980: Flags [P.], seq 1:141, ack 118, win 65535, length 140: HTTP: HTTP/1.1 200 OK
15:25:31.482869 IP 10.10.3.10.8080 > 10.10.1.10.64980: Flags [P.], seq 141:319, ack 118, win 65535, length 178: HTTP
15:25:31.483642 IP 10.10.1.10.64980 > 10.10.3.10.8080: Flags [.], ack 141, win 65535, length 0
15:25:31.484645 IP 10.10.1.10.64980 > 10.10.3.10.8080: Flags [.], ack 319, win 65535, length 0
15:25:31.484645 IP 10.10.1.10.64980 > 10.10.3.10.8080: Flags [F.], seq 118, ack 319, win 65535, length 0
15:25:31.484757 IP 10.10.3.10.8080 > 10.10.1.10.64980: Flags [.], ack 119, win 65535, length 0
15:25:31.484775 IP 10.10.3.10.8080 > 10.10.1.10.64980: Flags [F.], seq 319, ack 119, win 65535, length 0
15:25:31.485469 IP 10.10.1.10.64980 > 10.10.3.10.8080: Flags [.], ack 320, win 65535, length 0
```

Figure 11. `tcpdump -nn` capture of one full connection of our client.

What the capture shows. Sequence numbers are shown relative to each side's start, as `tcpdump` prints them.

1. *Handshake.* The client (10.10.1.10, port 64980) sends a SYN with `win 65535` and `mss 1460`. The server answers with a SYN-ACK acknowledging 1534331093 (the client's random initial number plus one) and its own `mss 1460`, and the client's ACK follows. The MSS option appears only on these two segments.
2. *Request.* `GET /index.html HTTP/1.1` travels as one 117-byte segment (`seq 1:118`), which the server ACKs immediately.
3. *Response.* The header (140 bytes, `seq 1:141`) and the 178-byte body (`seq 141:319`) go out as two segments. The client ACKs each one (`ack 141`, then `ack 319`).
4. *Teardown.* Once the whole body has arrived, the client sends a FIN (`seq 118`), the server ACKs it (`ack 119`) and sends its own FIN (`seq 319`), and the client's last ACK (`ack 320`) closes the connection.

The window field is 65535 in every segment, and every data segment and FIN is acknowledged at once.

> The whole exchange takes about 8 ms, and the handshake round trip is about 2.6 ms. There are no retransmissions, duplicates or reordering in this capture, so the recovery paths described below are not visible here. We exercised them separately in the namespace test described at the end.

How our TCP works

Everything lives in `src/tcp.py`, with sequence numbers compared modulo 2^{32}. The client never hard-codes its own address: before the handshake it asks the kernel's routing table which local address reaches the destination (a UDP `connect()`, which sends nothing), and that gives 10.10.1.10 towards the server. The connection object is built from three small components, each covering one of the three mechanisms below, so the connection itself only moves segments and timers around.

Retransmission (`RttEstimator` and the connection's timer). There is a single timer for the oldest unacknowledged byte. We time one segment at a time and smooth the samples the usual way, $srtt \leftarrow \frac{7}{8} srtt + \frac{1}{8} r$ and $rttvar \leftarrow \frac{3}{4} rttvar + \frac{1}{4} |srtt - r|$, giving $RTO = \min(srtt + \max(4\, rttvar, 200\text{ ms}),\ 0.9\text{ s})$. The first RTO is 0.9 s. A pending sample is thrown away whenever something is retransmitted, so retransmitted segments are never timed. When the timer fires we double the RTO (still capped at 0.9 s), move the send pointer back to the oldest unacknowledged byte and resend that segment, and the rest follows as the window opens. If data is in flight and nothing new is acknowledged for 10 s, we abort the connection. The third duplicate ACK triggers a fast retransmit. Before that, each of the first two duplicates lets one extra new segment out, so even a small window can generate three. SYN and SYN-ACK are retried every 0.9 s for up to 8 s, and a FIN is retried until it is ACKed, for at most 2 s.

Out-of-order buffering (`ReceiveQueue`). In-order data goes straight to the application buffer and the next expected sequence number advances. A segment that starts beyond it but inside our 65 535-byte window is parked in a dictionary keyed by its sequence number, keeping the longer copy if one arrives twice. Anything below the next expected number is trimmed away, so duplicates are delivered only once. When a gap is filled, we keep releasing parked segments that now start at or below the next expected number, trimming any overlap, until nothing more moves. A peer FIN is only consumed once everything before it has arrived. We send a cumulative ACK for every data segment, including duplicate and out-of-order ones, and these repeated ACKs are what drive the sender's fast retransmit.

Congestion control (`NewReno`). The data in flight never exceeds $\min(cwnd, \text{peer window})$.

- *Slow start:* the initial window is 10 MSS, and each new ACK adds the number of bytes it acknowledges, up to one MSS, while $cwnd < ssthresh$. We do not cap the increase at *ssthresh*, so the last step can overshoot it by less than one MSS.
- *Congestion avoidance:* each new ACK adds $\text{MSS}^2/cwnd$, which is about one MSS per RTT.
- *Fast recovery:* on the third duplicate ACK, $ssthresh = \max(\text{flight}/2,\ 2\,\text{MSS})$, computed from the amount of data in flight and not from *cwnd*, and $cwnd = ssthresh + 3\,\text{MSS}$ (the NewReno variant of RFC 6582), and we note the highest byte sent. Every further duplicate ACK inflates the window by one MSS. A partial ACK resends the next hole and deflates the window by the amount acknowledged, as NewReno does. A full ACK ends recovery with $cwnd = ssthresh$.
- *Timeout:* *ssthresh* is set as above, *cwnd* drops to one MSS, recovery is cancelled and we are back in slow start.

The controller only decides what to do and the connection carries it out, which keeps the window logic separate from sending and timers.

Sending in bursts. Our receiver acknowledges every segment, so a sender that releases one segment per ACK produces a perfectly smooth stream. The kernel flows we share a link with get delayed ACKs from their receivers and send in small bursts. At a tail-drop queue a smooth flow is hit by far fewer drops than a bursty one, and in our first measurements it took up to 2.3 times the throughput of the curl flows. We therefore hold new data back until the window has room for four full segments and then send them together. Tails shorter than four segments and windows below five segments are sent immediately, so small transfers and recovery after a timeout are not slowed down. The window rules are untouched, so the data in flight never exceeds the congestion window or the peer's window. We measured this on the namespace copy of the topology with a 10 Mbit/s bottleneck, 10 ms delay and a 30-packet queue, running 1, 2 and 3 flows of each kind at once (our client downloading from our server against curl downloading from `http.server`). The ratio of our throughput to curl's was about 1.3, 2.0 and 2.3 before this change, and over three runs after it it was 0.67–0.78 for one flow of each kind, 0.73–1.26 for two and 0.88–1.17 for three, with Jain's fairness index between 0.93 and 0.99.